LUCSYS ROBOTICS

STM32 Mobile Chassis Development Manual

ROS Educational Robot Platforms

Version	Date	Description
V1.0	2024-08-28	Initial release
V1.1	2025-02-19	Added V550-series chassis information

Technical documentation for development, integration, and maintenance

Preface

Welcome to the STM32 Mobile Chassis Development Manual. This manual provides a practical and detailed reference for developing with our ROS educational robot platforms. It is intended for robotics enthusiasts, students, educators, and research users.

The manual introduces the basic architecture of a ROS mobile chassis and then covers kinematic analysis, communication and control concepts, motor closed-loop control, firmware organization, and programming methods. Our goal is to help you connect the underlying theory with practical robot development and build a stable, reliable mobile platform.

The content applies to six common chassis types used in our educational robot family: two-wheel differential drive, Ackermann steering, Mecanum drive, omni-wheel drive, tracked drive, and four-wheel drive. Each chassis has different motion characteristics, and the relevant differences are explained throughout the manual.

To make full use of this manual, we recommend basic familiarity with robotics, ROS, C programming, and fundamental electrical/electronic engineering. For ROS-specific setup and usage, refer to the Ubuntu/ROS tutorials, ROS series tutorials, and ROS development materials included with the platform.

Contents

1. Introduction - 1.1 Mobile Chassis and ROS; 1.2 Chassis Coordinate Frame

2. Mobile Chassis Components and Functions - 2.1 Controller and Drivers; 2.2 Encoders, Motors, and Servo; 2.3 Wheels; 2.4 OLED; 2.5 Switches and Buttons; 2.6 IMU Zero Update; 2.7 Potentiometer; 2.8 LED and Buzzer

3. Chassis Types and Kinematic Analysis - 3.1 Kinematic Analysis

4. Chassis Motor Control - 4.1 Wheel Linear Speed; 4.2 Incremental PI Controller

5. Chassis Control Modes - App, PS2, RC, CAN, USART, ROS

6. System Flow and Architecture - 6.1 Hardware Architecture; 6.2 STM32 Firmware Flow; 6.3 Motor Control Flow

7. Firmware Programming - 7.1 Serial Programming; 7.2 SWD Programming

8. Important Notes - 8.1 Code; 8.2 Power Interfaces; 8.3 Motor Protection; 8.4 Battery Maintenance

1. Introduction

This chapter explains how the Robot Operating System (ROS) works together with the mobile chassis and defines the coordinate frame used by the chassis.

1.1 Mobile Chassis and ROS

ROS provides middleware that enables efficient communication and coordination among hardware and software components. Within this architecture, the mobile chassis is not only the robot's physical mobility platform; it is also a key communication node that exchanges data with the ROS environment through serial communication.

- Communication node: the chassis operates as part of the ROS system and sends/receives motion-related messages.
- Data integration: the chassis can integrate odometry and IMU data to provide real-time robot-state feedback for localization, navigation, and motion planning.
- Control execution: the chassis receives ROS commands, such as velocity and steering commands, and converts high-level control objectives into motor drive signals.

This modular integration makes it easy to add or replace ROS nodes as a project evolves without redesigning the entire chassis. In this manual, we describe the STM32-based mobile chassis used in our platforms, including its control architecture, data handling, and communication with an upper-level computer such as a ROS host.

1.2 Chassis Coordinate Frame

A coordinate frame is fundamental for describing robot position, orientation, and motion. Our ROS educational robot chassis use the coordinate convention illustrated below.

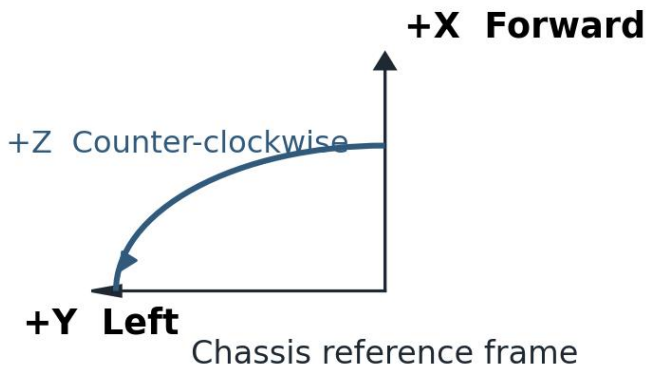


Figure 1-1. Chassis coordinate frame

The positive X-axis points forward, the positive Y-axis points to the left, and positive rotation about the Z-axis is counter-clockwise.

2. Mobile Chassis Components and Functions

A typical robot can be divided into three functional layers. The upper-level decision layer performs computationally intensive but less time-critical planning on a computer operating system. The perception layer uses sensors such as LiDAR, depth cameras, and inertial navigation sensors to observe the environment. The low-level execution layer has lower computational requirements but strict real-time requirements; it normally runs on a microcontroller and executes control commands while reporting robot status in real time. This chapter focuses on the low-level execution layer of the chassis.

- Typical chassis components include the controller board, servo (for Ackermann steering), encoder motors, wheels, power supply, and user interfaces.

2.1 Controller and Motor Drivers

Our mobile chassis controller board uses an STM32F407VET6 MCU and adopts an integrated control-and-drive architecture. The main functional interfaces are summarized below.

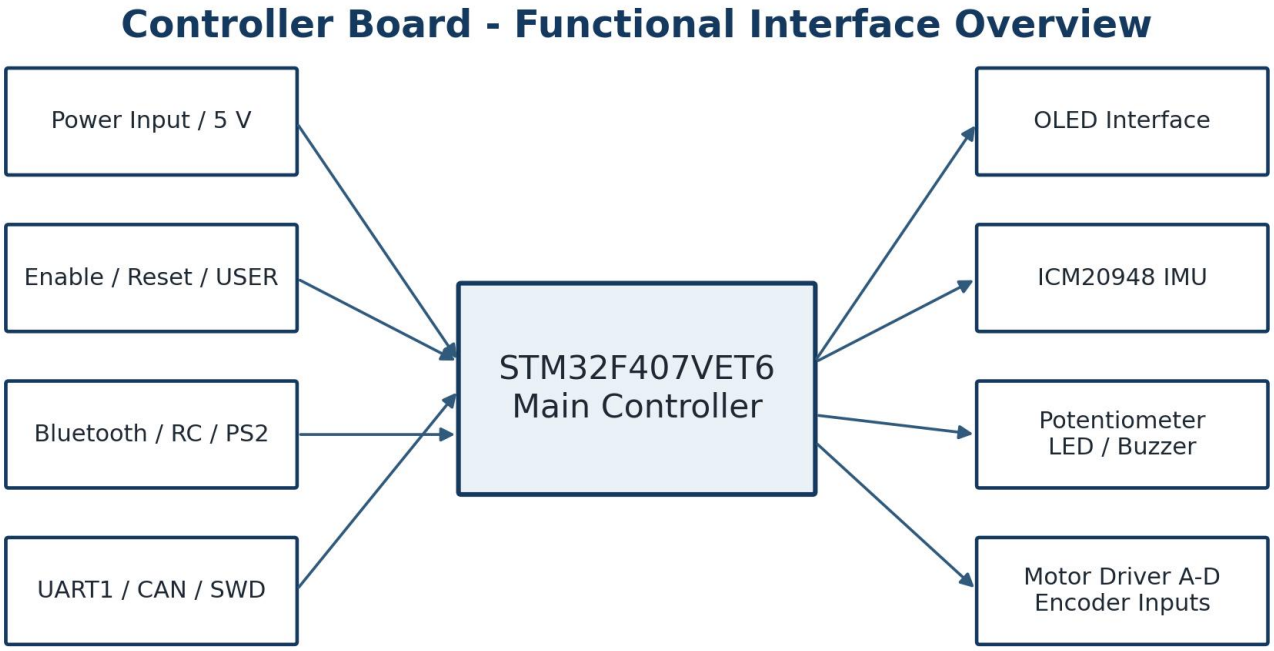


Figure 2-1. Functional overview of the STM32 controller board

Controller

The STM32F407VET6 is the core of the controller board. It receives commands from the upper-level computer and converts them into motor drive signals. Stable signal processing and real-time response are important so that motion commands can be executed accurately. The controller board also integrates an ICM20948 attitude sensor for real-time motion and attitude feedback to the host computer.

Motor drivers

The motor driver stage converts controller outputs into the current and voltage required by the motors. The integrated design places the motor driver circuitry on the same control board, simplifying the

electrical connections among controller, driver, motor, and encoder. This improves integration density and helps keep the chassis compact and easy to deploy.

Interfaces and expansion

The board provides multiple interfaces, switches, and peripheral connections for expansion. It also integrates an ICM20948 6-axis IMU and motor driver circuitry. Encoder-equipped motors can be connected to the corresponding motor interfaces. When integrating a third-party motor, confirm that the connector pinout matches the controller schematic before use.

The chassis supports multiple remote-control options, including mobile-app control, PS2 control, and RC transmitter control. Detailed wiring instructions and operating tutorials are included with the platform documentation.

2.2 Encoders, Motors, and Servo

This section introduces the basic operating principles of the motor, encoder, and servo used in our chassis. More detailed control examples are available in the STM32 firmware and motor-control tutorials included with the platform.

1) Encoders

Encoders measure and report motor speed and position. The chassis may use Hall encoders or GMR (giant magnetoresistance) encoders. Encoder feedback is essential for closed-loop control because it allows the motor controller to adjust the actual motor state to follow the command from the upper-level controller.

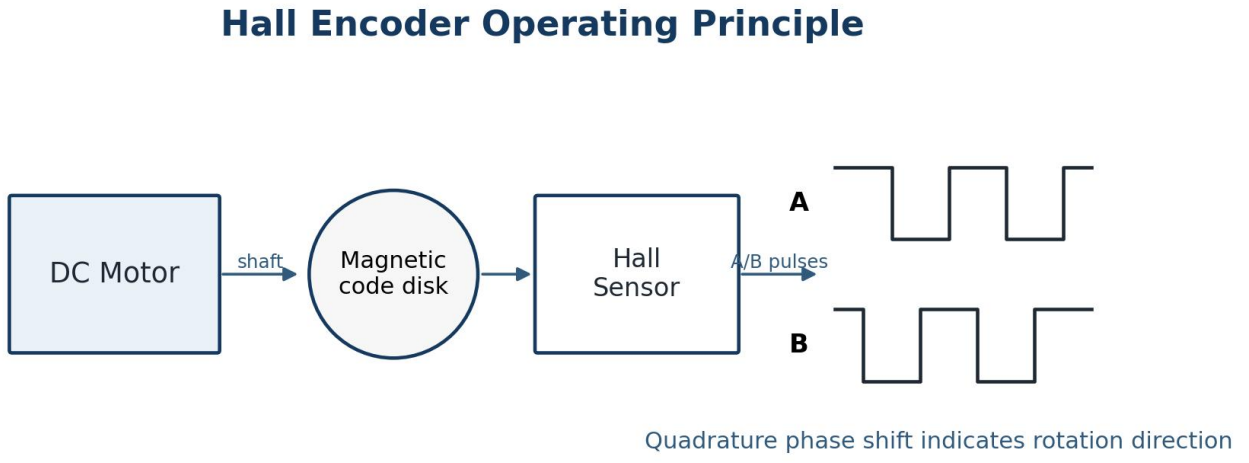


Figure 2-2. Hall encoder operating principle

A Hall encoder consists of a magnetic code disk and Hall sensing elements. The code disk rotates coaxially with the motor. As the motor turns, the Hall elements generate pulse signals. Two square-wave signals, phase A and phase B, have a phase offset; the MCU can use this quadrature relationship to determine rotation direction and calculate position and speed.

A GMR encoder measures displacement through the magnetoresistive effect. Typical GMR encoders provide A, B, and Z signals. A and B are quadrature position signals, while Z marks a once-per-revolution

reference position. In this implementation, only A and B are used. GMR encoders offer high resolution, high precision, and low temperature drift.

Encoder resolution is determined by line count. For example, a 13-line Hall encoder produces 13 pulses per code-disk revolution, while a 500-line GMR encoder produces 500 pulses per encoder-shaft revolution. Direction is determined from the phase relationship between A and B. For additional details, refer to the encoder usage and speed-measurement tutorial included with the STM32 firmware materials.

2) DC geared motor

The motor converts electrical energy into mechanical motion. Our ROS educational chassis use responsive DC motors with encoder feedback. The encoder continuously reports motor speed and position, enabling accurate speed and position control.

The gear ratio describes the relationship between the motor rotor speed and the output-shaft speed. For example, a 1:30 reduction means the motor rotor (input shaft) turns 30 revolutions for one revolution of the output shaft. The reduction gearbox provides lower speed and higher output torque.

For the MG513 brushed DC geared motor, the encoder is connected to the motor rotor (input shaft), while the output shaft is driven through the reduction gear train. Additional examples are available in the DC motor and TB6612 motor-control tutorial.

3) Servo

On the Ackermann chassis, a servo adjusts the steering angle. A servo integrates a DC motor, drive electronics, reduction gearing, and position feedback into a compact unit and can rotate accurately to a commanded angle using a simple control signal.

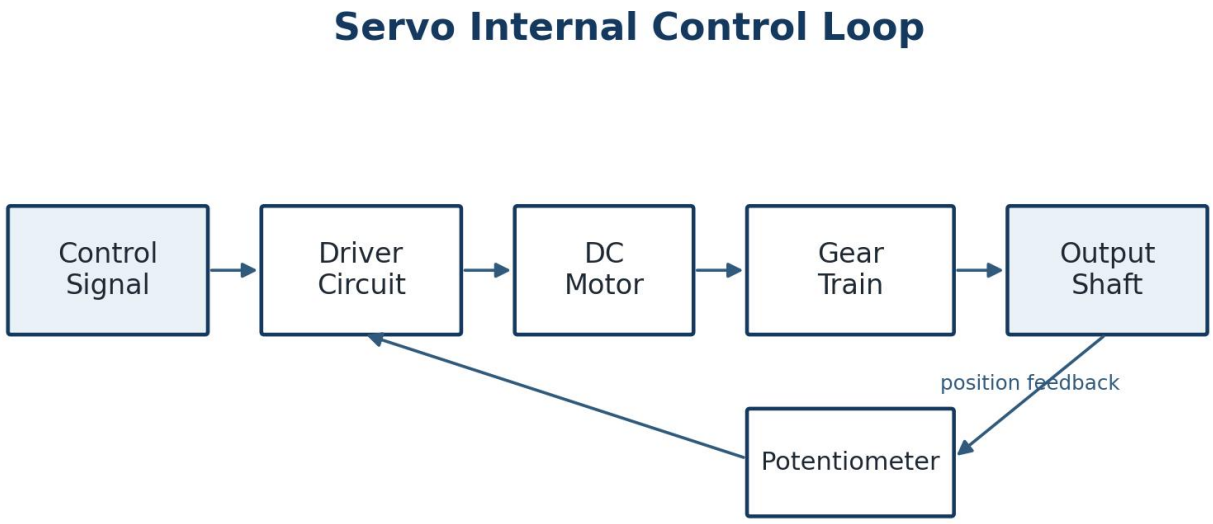


Figure 2-3. Simplified internal structure of a servo

The drive circuit receives the control signal and drives the motor. The gear train reduces speed and increases output torque. A potentiometer rotates with the final gear stage and measures the output-shaft angle. The internal circuit compares the measured angle with the target and drives the motor until the commanded position is reached and held.

2.3 Wheels

The wheels are the chassis components in direct contact with the ground, so wheel type and parameters have a direct effect on mobility. We select the wheel type according to the intended application. Wheel parameters affect stability, payload capacity, terrain passability, and the parameters used in kinematic analysis. Because different wheel and chassis structures lead to different kinematic models, wheel-speed calculation is discussed together with motor control in Chapter 4.

2.4 OLED Display Information

The controller board provides an OLED interface, and the OLED displays the current chassis state. The exact content varies slightly by chassis type. The following tables summarize the display meaning.

Chassis	OLED line content
Tracked	1: chassis type + Z-axis gyro zero-bias drift; 2: Z angular velocity after zero-bias removal; 3: left motor target/actual speed (mm/s); 4: right motor target/actual speed (mm/s); 5: motor A/B PWM; 6: control mode, enable state, battery voltage
Ackermann	1: chassis type + Z-axis gyro zero-bias drift; 2: corrected Z angular velocity; 3: left motor target/actual speed; 4: right motor target/actual speed; 5: servo command value; 6: control mode, enable state, battery voltage
Two-wheel differential	Same format as tracked chassis: gyro status, left/right target and actual speed, motor PWM, control mode/enable/battery voltage
Omni-wheel	1: chassis type + Z-axis gyro zero-bias drift; 2-4: motor A/B/C target and actual speed; 5: real-time angular velocity calculated from encoders; 6: control mode, enable state, battery voltage
Mecanum	1: chassis type + Z-axis gyro zero-bias drift; 2-5: motor A/B/C/D target and actual speed; 6: control mode, enable state, battery voltage
Four-wheel drive	1: chassis type + Z-axis gyro zero-bias drift; 2-5: motor A/B/C/D target and actual speed; 6: control mode, enable state, battery voltage
550-series Mecanum	Same six-line format as the standard Mecanum chassis, with the 550-series chassis identifier shown on line 1
550-series 4WD	Same six-line format as the standard 4WD chassis, with the 550-series chassis identifier shown on line 1

Control mode	CAN	Bluetooth App	PS2	RC transmitter	Serial 1	Serial 3
OLED indication	CAN	APP	PS2	R-C	USART	ROS

2.5 Enable Switch and Buttons

1) Enable switch

The enable switch is located on the controller board. Motor control is enabled only when the switch is ON. For startup stability, the firmware forces the motors to remain disabled for the first 10 seconds after the STM32 powers on. During this interval, the OLED may still show OFF even if the physical switch is ON. After approximately 10 seconds, the status updates to ON. During the same startup interval, Serial Port 1 and Serial Port 3 are configured not to receive data.

2) Reset button

Press the RESET button once to restart the STM32 controller.

3) USER button

Double-press the USER button to update the gyroscope zero point. See Section 2.6 for the purpose and use of this function.

2.6 IMU Zero-Point Update

During startup initialization, the chassis compensates the IMU zero drift. If significant map drift is observed during navigation or mapping, manually updating the gyroscope zero point may improve the result. To update it, keep the chassis stationary and double-press the USER button on the STM32 controller.

2.7 Chassis-Selection Potentiometer

A potentiometer on the controller board is used to select the chassis type. To simplify firmware management, the STM32 program supports multiple chassis configurations in one code base. Adjust the potentiometer to select the desired chassis; the selected type is shown on the first line of the OLED. After changing the potentiometer value, press RESET so that the new chassis setting is applied. This allows the chassis configuration to be changed without editing the firmware source code.

2.8 LED and Buzzer

Buzzer

The board includes a buzzer. Before use, confirm that it is connected to the A8 and ON pins as specified by the board configuration; otherwise the buzzer is not enabled. The buzzer emits a short tone at startup. When the battery voltage drops below 10.5 V, it gives the first low-voltage warning. When the voltage drops below 10.0 V, it beeps at approximately 1 Hz.

User LED

During the first 10 seconds after power-on, the firmware performs initialization, including IMU zero-drift sampling and calibration. The user LED flashes rapidly during this process. After initialization is complete, it returns to the normal flashing state.

3. Chassis Types and Kinematic Analysis

This chapter explains how the linear speeds of individual wheels relate to the chassis translational and rotational velocities. In a typical control flow, the upper-level computer sends a three-axis target velocity to the chassis. Kinematic analysis converts that command into target wheel speeds, and the motor controllers drive the wheels so the chassis follows the requested motion.

Chassis type	Forward / backward	Lateral motion	In-place rotation	Payload capacity	Turning friction
Mecanum	Yes	Yes	Yes	Relatively low	Relatively low
Omni-wheel	Yes	Yes	Yes	Relatively low	Relatively low
Two-wheel differential	Yes	No	Yes	Relatively low	Relatively low

Chassis type	Forward / backward	Lateral motion	In-place rotation	Payload capacity	Turning friction
Four-wheel drive	Yes	No	Yes	Relatively high	Relatively high
Tracked	Yes	No	Yes	Relatively low	Relatively high
Ackermann	Yes	No	No	Relatively high	Relatively low

The process of converting chassis target velocity into motor target velocity is referred to here as kinematic analysis. It includes forward and inverse kinematics:

- Forward kinematics: calculate the chassis X-, Y-, and Z-axis velocities from the individual wheel speeds.
- Inverse kinematics: calculate individual wheel speeds from the desired chassis X-, Y-, and Z-axis velocities.

3.1 Kinematic Analysis

Detailed derivations for the different chassis types are available in the Chassis Control and Kinematic Analysis tutorial included with the platform.

4. Chassis Motor Control

To make the chassis respond accurately to velocity commands and operate stably, the motors use closed-loop control. After kinematic analysis produces the wheel-speed targets, the motors must be driven so that the measured wheel speeds track those targets. Encoder feedback provides the motor speed needed to calculate real-time wheel linear velocity.

The firmware uses PID-based closed-loop motor control. PID refers to Proportional, Integral, and Derivative control. In this chassis implementation, the speed loop uses an incremental PI controller. This chapter focuses on wheel-speed calculation and the PI speed-control function.

4.1 Wheel Linear Speed Calculation

Encoder pulse counts are used to calculate motor speed and then wheel linear speed. The relationship is:

linear speed = encoder counts per second x wheel circumference / encoder resolution / motor reduction ratio

In the STM32 firmware, the corresponding calculation is implemented as:

```
MOTOR_A.Encoder = Encoder_A_pr * CONTROL_FREQUENCY * Wheel_perimeter / Encoder_precision;
```

- Encoder_A_pr: raw encoder count data.
- CONTROL_FREQUENCY: control frequency in Hz.
- Encoder_precision: encoder resolution, determined by motor mechanical structure and encoder device.
- Wheel_perimeter: wheel circumference in meters.
- MOTOR_A.Encoder: real-time linear speed of motor A wheel in m/s.

The example above shows motor A. The same calculation is applied to motors B, C, and D when present.

4.2 Incremental PI Controller

Kinematic analysis provides the target motor speed, while encoder processing provides the measured speed. Both values are sent to the speed controller so the actual speed converges toward the target. The firmware implements the following incremental PI controller:

```
/***************************************************************************
 * Function: Incremental PI speed control
 * Inputs:   Encoder – measured encoder speed
 *           Target  – target speed
 * Return:   Pwm     – motor PWM command
 * Formula:  pwm += Kp[e(k)–e(k–1)] + Ki*e(k)
 ***************************************************************************/
int Incremental_PI_A(float Encoder, float Target)
{
    static float Bias, Pwm, Last_bias;
    Bias = Target – Encoder;
    Pwm += Velocity_KP * (Bias – Last_bias) + Velocity_KI * Bias;

    if (Pwm > 7200)  Pwm = 7200;
    if (Pwm < –7200) Pwm = –7200;

    Last_bias = Bias;
    return Pwm;
}
```

For a deeper introduction to PID/PI control and the associated firmware examples, refer to the motor-control tutorials included with the platform.

5. Chassis Control Modes

The chassis supports several control modes: mobile-app control, PS2 controller, RC transmitter, CAN control, and serial control. Serial control is further divided into ROS mode and USART mode. After power-on, the current mode is shown at the lower-left of the OLED. The default startup mode is ROS control.

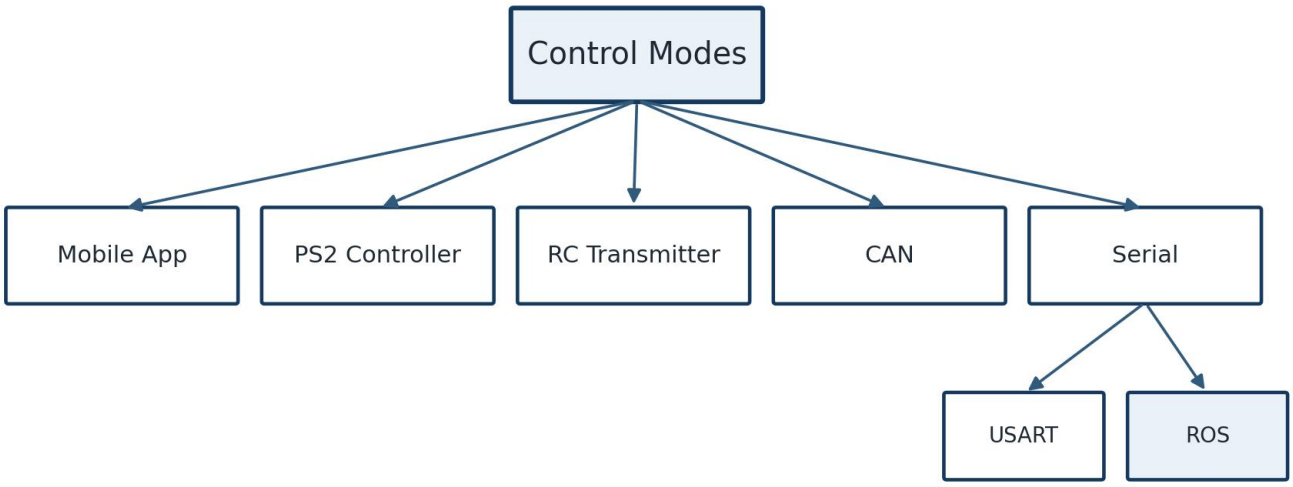


Figure 5-1. Supported chassis control modes

Detailed wiring instructions and operating tutorials for these control modes are included with the platform documentation.

6. System Flow and Architecture

The hardware and firmware architecture can be summarized using the following block diagrams. These diagrams show how the upper-level ROS computer, sensors, STM32 controller, user interfaces, and motors interact.

6.1 Robot Hardware Architecture

A typical system includes a ROS host computer, LiDAR, STM32 controller, encoder motors, integrated motor drivers, Bluetooth, PS2 controller, RC receiver, IMU, and expansion interfaces. Serial Port 1 and CAN are also reserved for user expansion.

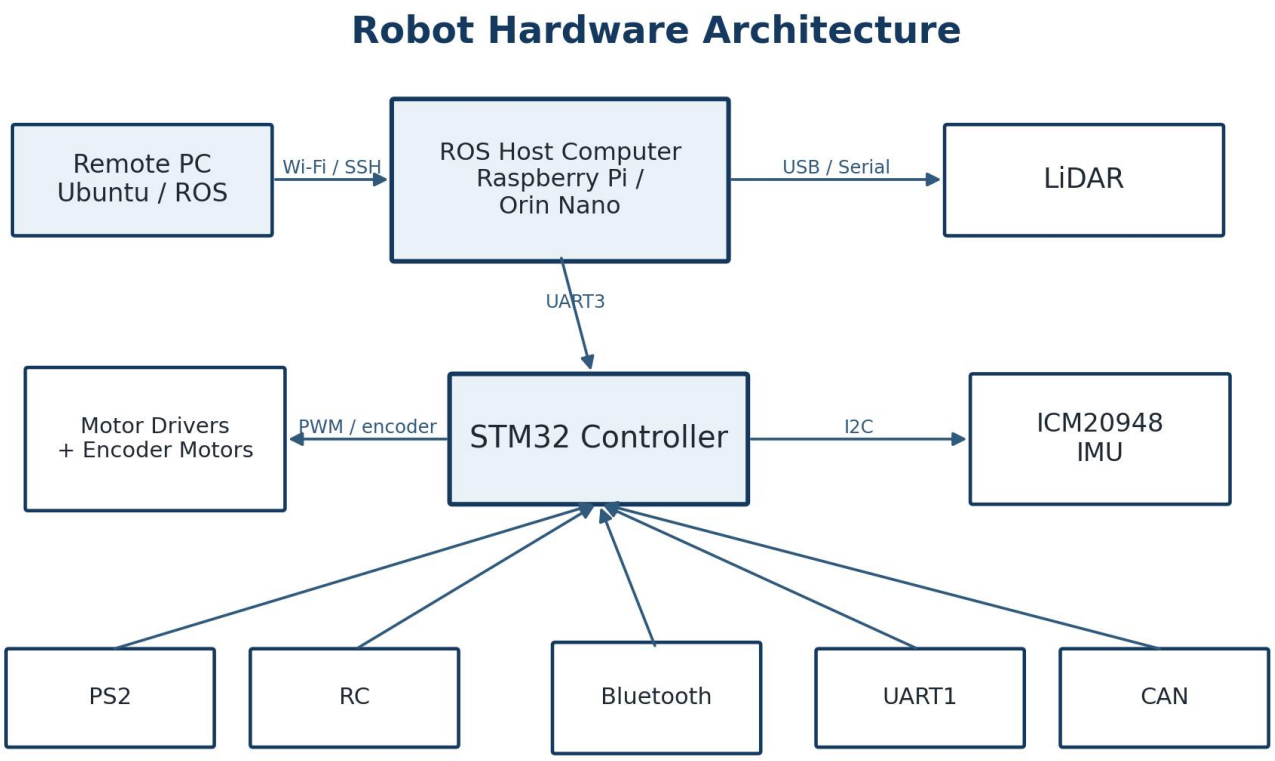


Figure 6-1. Robot hardware architecture

LiDAR is an important navigation and perception sensor. It measures the distance and shape of surrounding objects using reflected laser light and can provide environmental information for mapping, path planning, and obstacle avoidance.

6.2 STM32 Firmware Flow

The controller uses the STM32F407VET6 MCU. The firmware is written in C and developed in Keil MDK-ARM (Keil5). FreeRTOS is used to improve real-time response and organize concurrent activities such as sensor acquisition, motion control, display updates, and communication.

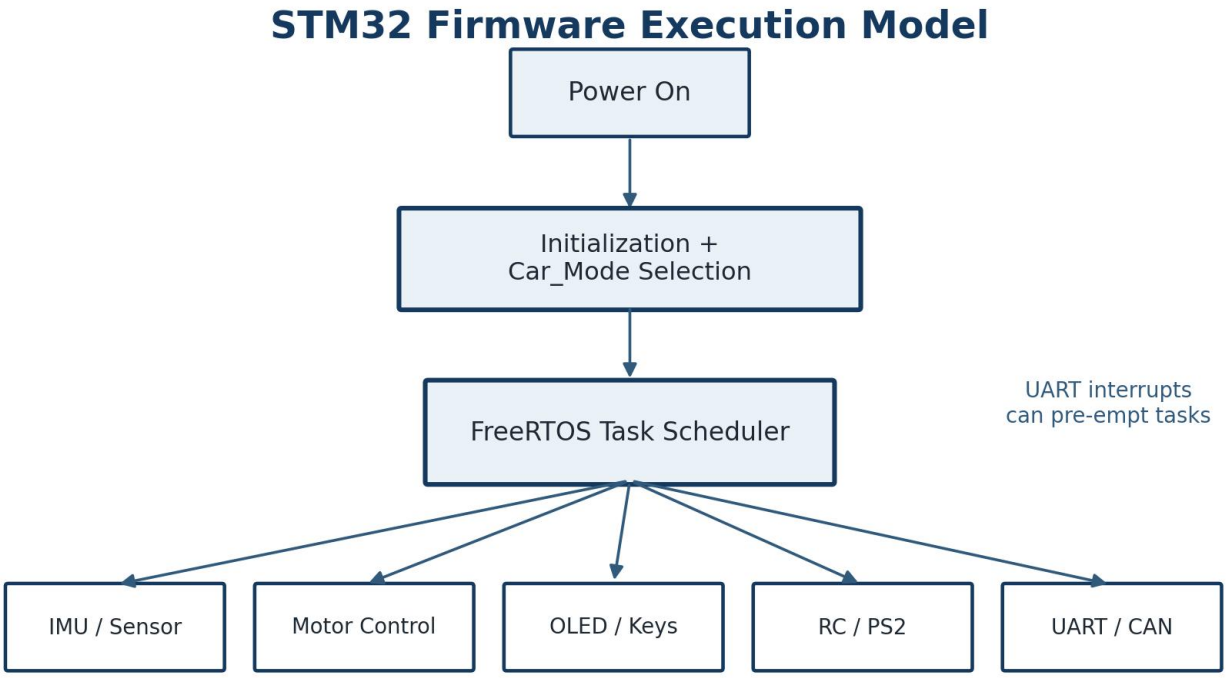


Figure 6-2. Simplified STM32 firmware execution flow

The FreeRTOS scheduler dynamically executes tasks according to their configured priorities. The visual order of tasks in a flow diagram should not be interpreted as the actual priority order; priority settings are defined in the source code. The system can also respond immediately to interrupts, such as Bluetooth control data on Serial Port 2 or ROS host data on Serial Port 3, preserving real-time behavior.

At power-on, initialization includes sampling the chassis-selection potentiometer and calculating Car_Mode. The selected Car_Mode determines which chassis-specific parameters and software branches are initialized.

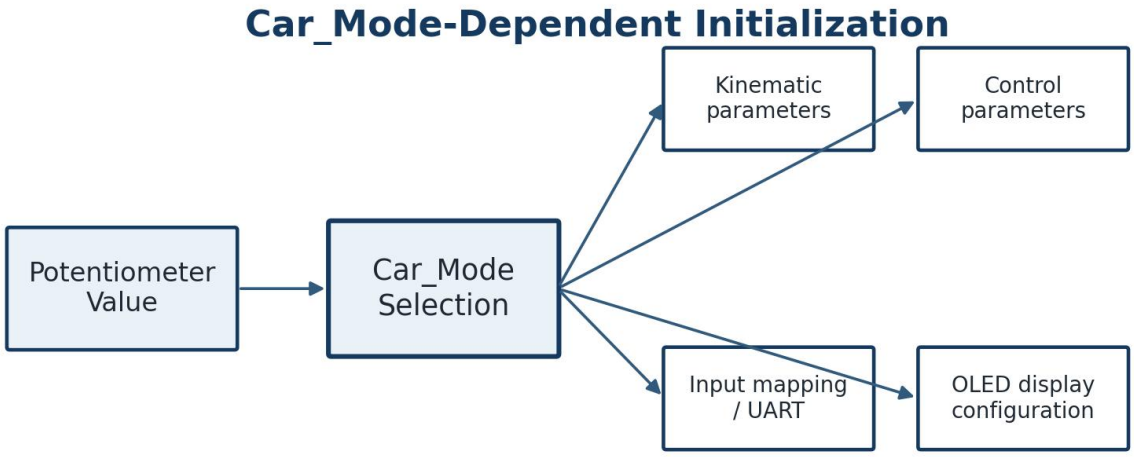


Figure 6-3. Car_Mode-dependent initialization

For deeper firmware-level study, refer to the STM32 source-code tutorials included with the platform.

6.3 Motor Control Flow

All supported control modes ultimately change the robot target velocity. Kinematic analysis converts that target into individual motor speed targets. The speed PI controller then generates the motor drive commands, while encoder feedback closes the loop.

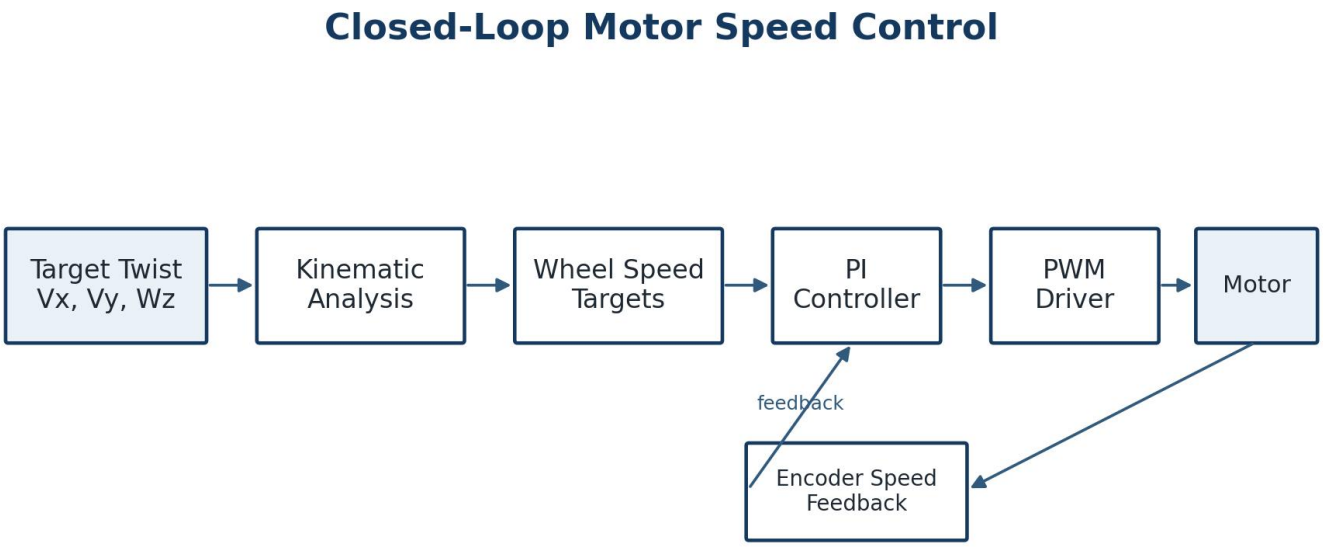


Figure 6-4. Motor control flow

Because the motor driver stage is integrated on the controller board, the DC motors only need to be connected to the corresponding motor interfaces on the board for normal use.

7. Firmware Programming

The STM32 controller can be programmed through the serial programming interface or the SWD interface. Serial programming uses the designated USB programming connection. For SWD, an ST-Link programmer is recommended and is normally prepared separately. The relevant firmware, tools, and drivers are included with the platform.

7.1 Serial Programming

The controller includes a one-click download circuit to simplify firmware programming. Use the USB data cable and the designated programming port fitted to your controller version.

1) Hardware preparation

- STM32 controller board
- USB data cable for the controller programming interface

2) Software preparation

Use the included FlyMCU programming utility and install the included USB-to-TTL driver. After installation, use the operating system Device Manager to confirm that the USB serial device is recognized without an error indicator.

3) Program the firmware

1. Open the included FlyMCU programming utility.
2. Select the detected COM port and the firmware image according to the programming instructions.
3. Start programming. If the option to execute after programming is enabled, the firmware will run automatically after the download completes.

7.2 SWD Programming

The controller also supports firmware download through the SWD interface. The SWD signals are identified on the board as PA13 and PA14.

1) Hardware preparation

- STM32 controller board
- ST-Link programmer

2) Software preparation

Install the driver for the ST-Link (or J-Link, if used). Confirm in Device Manager that the programmer is detected correctly.

3) Wiring

Programmer	STM32 controller
SWDIO	PA13
SWCLK	PA14
GND	GND

4) Download the program

Use the programming command in Keil/MDK to download the firmware. When “run after programming” is enabled, the program starts automatically after download. The default project configuration is set for ST-Link; using J-Link requires the corresponding MDK debug/programming configuration to be changed.

8. Important Notes

8.1 Code and RTOS Behavior

The STM32 firmware is organized around an RTOS task model rather than a purely interrupt-driven program. Higher-priority tasks are scheduled ahead of lower-priority tasks, while interrupts still have priority over RTOS tasks. If a task contains a blocking logic error, execution may become stuck at that point. For example, if a Serial Port 3 transmit task runs but Serial Port 3 was not initialized correctly, the program may stop progressing when that task executes. If the firmware appears to hang during debugging, inspect the tasks one by one for blocking errors or bugs.

8.2 Power Interfaces on the Controller Board

The controller provides 5 V and 3.3 V output pins for external devices, but these outputs should not be overloaded. The 5 V output is not recommended for loads above 1.5 A, and the 3.3 V output is not

recommended for loads above 200 mA. The controller schematic shows two 5 V power circuits on the adapter board: one for basic peripherals and a separate one for the ROS host computer such as a Raspberry Pi or Orin Nano via the designated USB power output.

8.3 Motor Use and Protection

Avoid motor stall conditions because sustained stall current can damage the controller board. Before the robot has been fully tested, raise the chassis so the wheels are off the ground. This reduces the risk of damage from unexpected motor motion during firmware or control debugging.

8.4 Battery Use and Maintenance

The OLED displays battery voltage. Charge the battery promptly when the voltage falls below 10.8 V. The battery includes over-discharge and over-charge protection. Do not operate the robot from the battery while the battery is being charged. Connect the charger to the battery charging interface specified for the platform.